

Jogo da Velha

Distribuído

Arquitetura Cliente-Servidor com Sockets e Threads

Disciplina: Redes de Computadores - UFAL 2026.1

Equipe 3:

Vitor Samyr | Lucas Gabriel | Thomaz Xavier | Gabriel Damião | Luiz Henrique

O Desafio Tecnológico

Como conectar computadores diferentes em uma única partida simultânea?



| Pilares Tecnológicos



Sockets

Interface fundamental do S.O. para permitir que programas em máquinas distintas troquem fluxos de bytes via IP/Porta.



TCP/IPv4

Adoção do TCP para criar túneis confiáveis e orientados à conexão, garantindo a entrega sequencial de cada jogada.



Threads

Uso de concorrência para impedir o bloqueio de tela. O sistema lida com o teclado e a rede ao mesmo tempo.

O Modelo Cliente-Servidor

O Servidor (Autoridade)

Responsabilidades:

- ▶ Detentor do estado oficial do tabuleiro.
- ▶ Valida e proíbe jogadas irregulares.
- ▶ Orquestra o sistema de turnos (X vs O).
- ▶ Distribui ("Broadcast") a versão oficial do jogo.

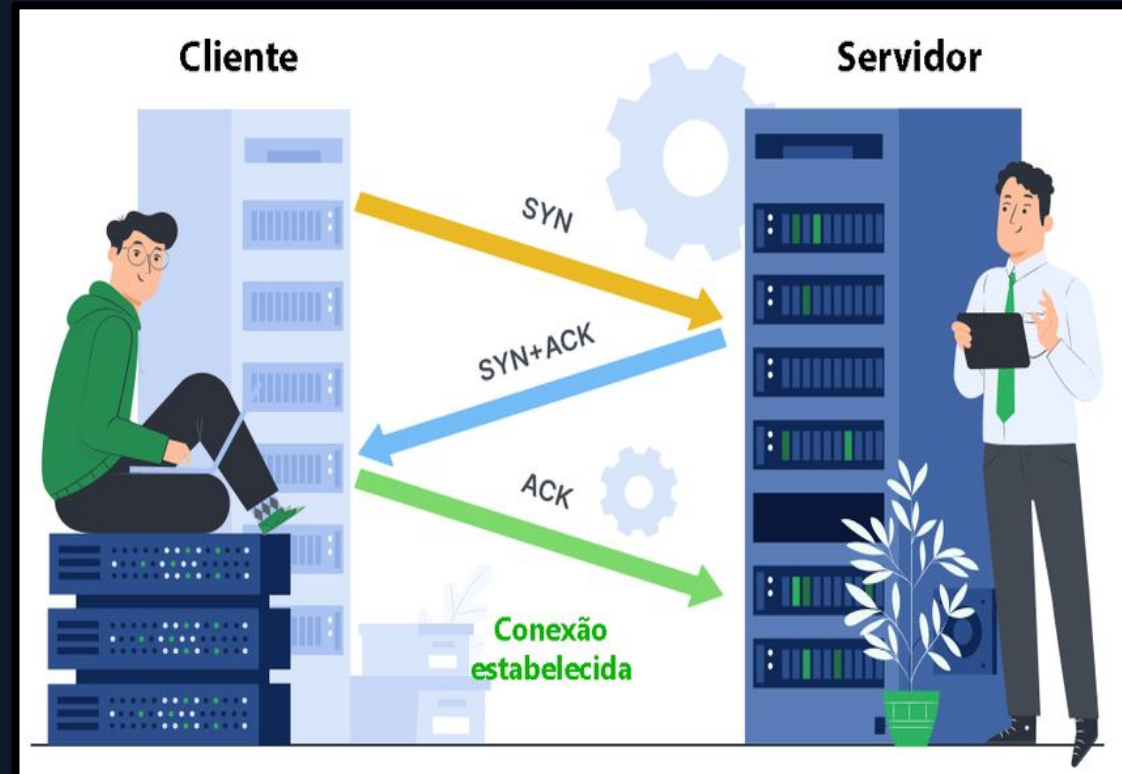
Os Clientes (Nós)

Responsabilidades:

- ▶ Conectam via IP Local ou Externo.
- ▶ Atuam como ponte de Input/Output.
- ▶ Capturam decisões do teclado humano.
- ▶ Renderizam as mensagens enviadas pelo servidor.

TCP: A Escolha do Protocolo

- **Ordem e Cronologia:** O Jogo da Velha baseia-se em turnos estritos. O TCP garante que as mensagens não invertam a ordem no tráfego.
- **Confiabilidade (Three-Way Handshake):** Garante que os pacotes cheguem ao destino e retransmite caso haja perda na rota.
- **Risco do UDP:** Se usássemos UDP, uma jogada perdida na rede causaria dessincronização invisível do tabuleiro entre os dois jogadores.



| Controle de Concorrência

1

Lock (Mutex) Global

Prevenção de Condição de Corrida

O Servidor gerencia múltiplas **Threads** simultâneas (uma para cada cliente conectado).

Para evitar que duas requisições sobreponham e corrompam o mesmo array de posições na memória RAM, utilizamos a trava `threading.Lock()`, garantindo a integridade dos dados durante a validação.

| Desafios Superados



Concorrência no I/O do Cliente: A função `input()` travava a leitura da rede. Solucionado criando uma "Daemon Thread" secundária exclusiva para escutar o servidor.



Codificação de Tipos: Entender que pela rede não trafegam "variáveis", mas sim fluxos de bytes, exigindo uso constante de `.encode()` e `.decode()`.



Validação de Regras: Lidar com inputs inválidos (letras, fora do range 1–9, posições já preenchidas) sem crashar o processo na porta do servidor.

Obrigado pela atenção!

Momento para dúvidas.